

Eventos en JavaScript

Introducción

Los eventos son acciones que ocurren en el navegador web, como hacer clic en un botón, mover el ratón, presionar una tecla, o cargar una página. JavaScript nos permite "escuchar" estos eventos y ejecutar código en respuesta a ellos, creando páginas web interactivas.

Conceptos Fundamentales

¿Qué son los eventos?

Los eventos son señales que indican que algo ha sucedido en el documento HTML. Pueden ser:

- **Eventos de usuario:** clic, movimiento del ratón, pulsación de teclas
- **Eventos del navegador:** carga de página, redimensionado de ventana
- **Eventos de formulario:** envío, cambio de valores
- **Eventos de multimedia:** reproducción, pausa de video/audio

Flujo de eventos

Cuando ocurre un evento, este sigue un flujo a través del DOM:

1. **Fase de captura:** El evento viaja desde la raíz hasta el elemento objetivo
2. **Fase objetivo:** El evento llega al elemento que lo generó
3. **Fase de burbujeo:** El evento viaja desde el elemento objetivo hacia la raíz

```
<div id="padre">
  <div id="hijo">
    <button id="boton">Clic aquí</button>
  </div>
</div>
```



Formas de Manejar Eventos

Existen varias formas de asociar eventos a elementos en JavaScript:

Atributos HTML (no recomendado)

Es posible definir eventos directamente en el HTML usando atributos, pero no es una práctica recomendada por razones de mantenimiento y separación de responsabilidades. Es mejor usar JavaScript para manejar eventos, y definir en el HTML solo la estructura.

```
<button onclick="alert('¡Hola!')">Clic aquí</button>
<button onmouseover="cambiarColor(this)">Pasa el ratón</button>
```



Propiedades de JavaScript

Otra forma es asignar una función a la propiedad del evento del elemento. Sin embargo, esta técnica solo permite un manejador por evento.

```
const boton = document.getElementById('miBoton');

boton.onclick = function() {
  alert('¡Botón clickeado!');
};

// O con función flecha
boton.onclick = () => {
  alert('¡Botón clickeado!');
};
```



addEventListener (recomendado)

La forma más flexible y recomendada de manejar eventos es usando el método `addEventListener`, que permite agregar múltiples manejadores para el mismo evento y ofrece más control sobre la propagación del evento.

```
const boton = document.getElementById('miBoton');

boton.addEventListener('click', function() {
  alert('¡Botón clickeado!');
});

// Con función flecha
boton.addEventListener('click', () => {
  alert('¡Botón clickeado!');
});

// Función externa
```



```
function manejarClic() {  
    alert('¡Botón clickeado!');  
}  
  
boton.addEventListener('click', manejarClic);
```

Principales Tipos de Eventos

Eventos de Mouse

```
const elemento = document.getElementById('miElemento');  
  
// Clic con botón izquierdo  
elemento.addEventListener('click', (e) => {  
    console.log('Clic en coordenadas:', e.clientX, e.clientY);  
});  
  
// Doble clic  
elemento.addEventListener('dblclick', () => {  
    console.log('Doble clic');  
});  
  
// Botón del mouse presionado  
elemento.addEventListener('mousedown', (e) => {  
    console.log('Botón presionado:', e.button); // 0=izq, 1=medio, 2=der  
});  
  
// Botón del mouse liberado  
elemento.addEventListener('mouseup', () => {  
    console.log('Botón liberado');  
});  
  
// Ratón entra en el elemento  
elemento.addEventListener('mouseenter', () => {  
    elemento.style.backgroundColor = 'lightblue';  
});  
  
// Ratón sale del elemento  
elemento.addEventListener('mouseleave', () => {  
    elemento.style.backgroundColor = '';  
});  
  
// Movimiento del ratón sobre el elemento  
elemento.addEventListener('mousemove', (e) => {
```



```
    console.log('Posición:', e.offsetX, e.offsetY);
  });
```

Eventos de Teclado

```
// Escuchar en todo el documento
document.addEventListener('keydown', (e) => {
  console.log('Tecla presionada:', e.key);
  console.log('Código:', e.code);

  // Detectar teclas especiales
  if (e.key === 'Enter') {
    console.log('Enter presionado');
  }

  if (e.ctrlKey && e.key === 's') {
    e.preventDefault(); // Prevenir guardar del navegador
    console.log('Ctrl+S presionado');
  }
});

// Tecla liberada
document.addEventListener('keyup', (e) => {
  console.log('Tecla liberada:', e.key);
});

// Solo cuando se escribe un carácter
document.addEventListener('keypress', (e) => {
  console.log('Carácter:', e.key);
});

// En un input específico
const input = document.getElementById('miInput');
input.addEventListener('keydown', (e) => {
  if (e.key === 'Enter') {
    console.log('Enter en el input:', input.value);
  }
});
```



Eventos de Formulario

```
const formulario = document.getElementById('miFormulario');
const input = document.getElementById('miInput');
```



```

// Envío del formulario
formulario.addEventListener('submit', (e) => {
    e.preventDefault(); // Prevenir envío por defecto
    console.log('Formulario enviado');

    // Validar y procesar datos
    const datos = new FormData(formulario);
    console.log('Datos:', Object.fromEntries(datos));
});

// Cambio en input
input.addEventListener('change', (e) => {
    console.log('Valor cambiado a:', e.target.value);
});

// Escribir en input (tiempo real)
input.addEventListener('input', (e) => {
    console.log('Escribiendo:', e.target.value);
});

// Foco en elemento
input.addEventListener('focus', () => {
    console.log('Input enfocado');
    input.style.backgroundColor = 'lightyellow';
});

// Perder foco
input.addEventListener('blur', () => {
    console.log('Input desenfocado');
    input.style.backgroundColor = '';
});

// Para selects
const select = document.getElementById('miSelect');
select.addEventListener('change', (e) => {
    console.log('Opción seleccionada:', e.target.value);
});

```

Eventos de Ventana

```

// Carga completa de la página
window.addEventListener('load', () => {
    console.log('Página completamente cargada');
});

```



```
// DOM cargado (sin esperar imágenes)
document.addEventListener('DOMContentLoaded', () => {
  console.log('DOM listo');
});

// Redimensionar ventana
window.addEventListener('resize', () => {
  console.log('Ventana redimensionada:', window.innerWidth, window.innerHeight);
});

// Scroll
window.addEventListener('scroll', () => {
  console.log('Scroll vertical:', window.pageYOffset);
});

// Antes de cerrar página
window.addEventListener('beforeunload', (e) => {
  e.preventDefault();
  e.returnValue = '¿Estás seguro de que quieres salir?';
});
```

El Objeto Event

El objeto `Event` proporciona información sobre el evento que ocurrió y métodos para controlar su comportamiento.

Propiedades principales

En este ejemplo, mostramos algunas propiedades útiles del objeto `Event`:

```
elemento.addEventListener('click', (event) => {
  // Información básica
  console.log('Tipo de evento:', event.type);
  console.log('Elemento objetivo:', event.target);
  console.log('Elemento actual:', event.currentTarget);

  // Coordenadas del mouse
  console.log('Coordenadas de pantalla:', event.screenX, event.screenY);
  console.log('Coordenadas de cliente:', event.clientX, event.clientY);
  console.log('Coordenadas de página:', event.pageX, event.pageY);
  console.log('Coordenadas del elemento:', event.offsetX, event.offsetY);
});
```



```
// Teclas modificadoras
console.log('Ctrl presionado:', event.ctrlKey);
console.log('Shift presionado:', event.shiftKey);
console.log('Alt presionado:', event.altKey);

// Timestamp
console.log('Momento del evento:', event.timeStamp);
});
```

Métodos del objeto Event

El objeto `Event` proporciona varios métodos para controlar el comportamiento del evento:

`preventDefault()`

Cancela la acción por defecto asociada al evento. Esto es útil cuando queremos reemplazar el comportamiento estándar del navegador con nuestro propio código.

¿Cuándo usar `preventDefault()`?

- **Formularios:** Evitar que se envíe el formulario automáticamente
- **Enlaces:** Prevenir la navegación a otra página
- **Teclas:** Evitar atajos de teclado del navegador
- **Comportamientos del navegador:** Impedir scroll, zoom, etc.

Ejemplo 1: Validar y prevenir envío de formulario

```
const formulario = document.getElementById('miFormulario');

formulario.addEventListener('submit', (e) => {
  // Prevenir el envío automático del formulario
  e.preventDefault();

  // Aquí podemos validar los datos
  const email = document.getElementById('email').value;

  if (!email.includes('@')) {
    alert('Email inválido');
    return; // No continuar si no es válido
  }

  // Si todo es válido, podemos enviar manualmente
  console.log('Formulario válido, enviando datos...');
```



```
// formulario.submit(); // Envío manual si es necesario
});
```

Ejemplo 2: Enlace personalizado sin navegación

```
const enlace = document.getElementById('miEnlace');

enlace.addEventListener('click', (e) => {
  // Evitar que se navegue a la URL del href
  e.preventDefault();

  console.log('Se hizo clic en el enlace, pero no se navega');

  // Hacer algo personalizado
  abrirDialogoPersonalizado();
});
```



Ejemplo 3: Atajos de teclado personalizados

```
document.addEventListener('keydown', (e) => {
  // Ctrl+S: Guardador personalizado
  if (e.ctrlKey && e.key === 's') {
    e.preventDefault(); // Evitar "Guardar como" del navegador
    guardarDatos();
  }

  // Ctrl+Q: Función personalizada
  if (e.ctrlKey && e.key === 'q') {
    e.preventDefault();
    mostrarMenuSalida();
  }
});
```



Nota importante: `preventDefault()` solo funciona si el evento es **cancelable** (tiene propiedad `cancelable: true`). Algunos eventos como `load` no se pueden cancelar.

```
elemento.addEventListener('click', (e) => {
  if (e.cancelable) {
    e.preventDefault();
  }
});
```



`stopPropagation()`

Detiene la propagación del evento hacia elementos padres (evita el burbujeo).

```
const hijo = document.getElementById('hijo');
const padre = document.getElementById('padre');

// Clic en el hijo
hijo.addEventListener('click', (e) => {
  e.stopPropagation(); // EL evento no sube al padre
  console.log('Clic en hijo');
});

// Clic en el padre
padre.addEventListener('click', (e) => {
  console.log('Clic en padre'); // Esto NO se ejecuta si hicimos clic en hijo
});
```

Diferencia entre `stopPropagation()` y `preventDefault()`:

- `stopPropagation()`: Evita que el evento suba a elementos padres, pero el comportamiento por defecto **SÍ ocurre**
- `preventDefault()`: Cancela el comportamiento por defecto del navegador, pero el evento **SÍ se propaga**

```
// Ejemplo con ambos métodos
const enlace = document.getElementById('miEnlace');
const padre = document.querySelector('nav');

enlace.addEventListener('click', (e) => {
  e.preventDefault(); // No navega a La URL
  e.stopPropagation(); // No sube al padre
  console.log('Solo se ejecuta este evento');
});

padre.addEventListener('click', (e) => {
  console.log('Evento padre'); // No se ejecutará
});
```

`stopImmediatePropagation()`

Detiene tanto la propagación como otros listeners del mismo elemento para el mismo evento.

```
const boton = document.getElementById('miBoton');

// Primer Listener
boton.addEventListener('click', (e) => {
    e.stopImmediatePropagation();
    console.log('Primer listener');
});

// Segundo Listener
boton.addEventListener('click', (e) => {
    console.log('Segundo listener'); // NO se ejecuta
});

// Listener en padre
document.addEventListener('click', (e) => {
    console.log('Listener padre'); // NO se ejecuta
});
```



Resumen de métodos:

```
elemento.addEventListener('click', (e) => {
    // preventDefault()
    // - Cancela acción por defecto
    // - El evento se sigue propagando
    // - Se puede cancelar solo si cancelable=true
    e.preventDefault();

    // stopPropagation()
    // - Evita que suba a elementos padres
    // - La acción por defecto SÍ ocurre
    // - Los listeners del mismo elemento SÍ se ejecutan
    e.stopPropagation();

    // stopImmediatePropagation()
    // - Evita propagación a padres
    // - Evita otros listeners del mismo elemento
    // - La acción por defecto SÍ ocurre
    e.stopImmediatePropagation();
});
```



Casos de Uso Prácticos

Validación de formulario en tiempo real

```
<form id="formularioRegistro">
  <input type="email" id="email" placeholder="Email" required>
  <div id="errorEmail" class="error"></div>

  <input type="password" id="password" placeholder="Contraseña" required>
  <div id="errorPassword" class="error"></div>

  <input type="password" id="confirmPassword" placeholder="Confirmar contraseña" required>
  <div id="errorConfirmPassword" class="error"></div>

  <button type="submit">Registrarse</button>
</form>
```

```
// Validación en tiempo real
const form = document.getElementById('formularioRegistro');
const email = document.getElementById('email');
const password = document.getElementById('password');
const confirmPassword = document.getElementById('confirmPassword');

// Validar email
email.addEventListener('input', () => {
  const errorDiv = document.getElementById('errorEmail');
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

  if (email.value && !emailRegex.test(email.value)) {
    errorDiv.textContent = 'Email inválido';
    errorDiv.style.color = 'red';
  } else {
    errorDiv.textContent = '';
  }
});

// Validar contraseña
password.addEventListener('input', () => {
  const errorDiv = document.getElementById('errorPassword');

  if (password.value.length < 8) {
    errorDiv.textContent = 'La contraseña debe tener al menos 8 caracteres';
    errorDiv.style.color = 'red';
  } else {
    errorDiv.textContent = 'Contraseña válida';
    errorDiv.style.color = 'green';
  }
});
```

```

});

// Confirmar contraseña
confirmPassword.addEventListener('input', () => {
    const errorDiv = document.getElementById('errorConfirmPassword');

    if (confirmPassword.value !== password.value) {
        errorDiv.textContent = 'Las contraseñas no coinciden';
        errorDiv.style.color = 'red';
    } else {
        errorDiv.textContent = 'Contraseñas coinciden';
        errorDiv.style.color = 'green';
    }
});

// Envío del formulario
form.addEventListener('submit', (e) => {
    e.preventDefault();

    // Validación final
    if (validarFormulario()) {
        console.log('Formulario válido, enviando...');
        // Aquí enviarías los datos
    } else {
        console.log('Formulario inválido');
    }
});

function validarFormulario() {
    // Implementar validación completa
    return email.value && password.value.length >= 8 && password.value === confi
}

```

Galería de imágenes interactiva

```

<div class="galeria">
  <div class="miniaturas">
    
    
    
  </div>
  <div class="visor">
    
  </div>
</div>

```



```
</div>
</div>
```

```
// Galería interactiva
const galeria = document.querySelector('.galeria');
const imagenPrincipal = document.getElementById('imagenPrincipal');

// Delegación de eventos para miniaturas
galeria.addEventListener('click', (e) => {
  if (e.target.classList.contains('miniatura')) {
    // Remover clase activa de todas las miniaturas
    document.querySelectorAll('.miniatura').forEach(img => {
      img.classList.remove('activa');
    });

    // Agregar clase activa a la seleccionada
    e.target.classList.add('activa');

    // Cambiar imagen principal con efecto fade
    imagenPrincipal.style.opacity = '0';

    setTimeout(() => {
      imagenPrincipal.src = e.target.dataset.full;
      imagenPrincipal.style.opacity = '1';
    }, 200);
  }
});

// Navegación con teclado
document.addEventListener('keydown', (e) => {
  const miniaturas = Array.from(document.querySelectorAll('.miniatura'));
  const activa = document.querySelector('.miniatura.activa');
  let indice = miniaturas.indexOf(activa);

  if (e.key === 'ArrowRight') {
    indice = (indice + 1) % miniaturas.length;
    miniaturas[indice].click();
  } else if (e.key === 'ArrowLeft') {
    indice = (indice - 1 + miniaturas.length) % miniaturas.length;
    miniaturas[indice].click();
  }
});
```

```
<div id="todoApp">
  <input type="text" id="nuevaTarea" placeholder="Nueva tarea...">
  <button id="agregarTarea">Agregar</button>
  <ul id="listaTareas"></ul>
</div>
```



```
// To-Do List con eventos
class TodoApp {
  constructor() {
    this.tareas = [];
    this.inicializarEventos();
  }

  inicializarEventos() {
    const input = document.getElementById('nuevaTarea');
    const botonAgregar = document.getElementById('agregarTarea');
    const lista = document.getElementById('listaTareas');

    // Agregar tarea con botón o Enter
    botonAgregar.addEventListener('click', () => this.agregarTarea());
    input.addEventListener('keydown', (e) => {
      if (e.key === 'Enter') this.agregarTarea();
    });

    // Delegación para botones de tareas
    lista.addEventListener('click', (e) => {
      const li = e.target.closest('li');
      const id = parseInt(li.dataset.id);

      if (e.target.classList.contains('completar')) {
        this.completarTarea(id);
      } else if (e.target.classList.contains('eliminar')) {
        this.eliminarTarea(id);
      }
    });
  }

  agregarTarea() {
    const input = document.getElementById('nuevaTarea');
    const texto = input.value.trim();

    if (texto) {
      const tarea = {
        id: Date.now(),
```



```

        texto: texto,
        completada: false
    };

    this.tareas.push(tarea);
    this.renderizar();
    input.value = '';
}
}

completarTarea(id) {
    const tarea = this.tareas.find(t => t.id === id);
    if (tarea) {
        tarea.completada = !tarea.completada;
        this.renderizar();
    }
}

eliminarTarea(id) {
    this.tareas = this.tareas.filter(t => t.id !== id);
    this.renderizar();
}

renderizar() {
    const lista = document.getElementById('listaTareas');
    lista.innerHTML = '';

    this.tareas.forEach(tarea => {
        const li = document.createElement('li');
        li.dataset.id = tarea.id;
        li.className = tarea.completada ? 'completada' : '';

        li.innerHTML = `
            <span class="texto">${tarea.texto}</span>
            <button class="completar">${tarea.completada ? 'Desmarcar' : 'Completar'}</button>
            <button class="eliminar">Eliminar</button>
        `;

        lista.appendChild(li);
    });
}

// Inicializar aplicación
document.addEventListener('DOMContentLoaded', () => {

```

```
new TodoApp();
});
```

`removeEventListener`

Para optimizar la memoria, es importante remover event listeners cuando no se necesiten:

```
// Función nombrada para poder removerla después
function manejarClic() {
  console.log('Clic manejado');
}

const boton = document.getElementById('miBoton');

// Agregar Listener
boton.addEventListener('click', manejarClic);

// Remover Listener
boton.removeEventListener('click', manejarClic);

// Para funciones anónimas, guardar referencia
const manejarHover = (e) => {
  console.log('Hover');
};

elemento.addEventListener('mouseenter', manejarHover);
// ...más tarde...
elemento.removeEventListener('mouseenter', manejarHover);
```